

Data Mining

BAYESIAN MODELS

BY: IVETA MRÁZOVÁ

Bayesian Classification

- Bayesian formula for the computation of the conditional probability

~ probability, that hypothesis ***H*** is true, if we observe ***E***

$$P(H | E) = \frac{P(E | H) P(H)}{P(E)}$$

- ***P(H)*** apriori probability of the hypothesis ***H***
- ***P(H|E)*** a posteriori (conditional) probability
- ***P(E)*** probability of the event ***E***

Bayesian Classification (2)

→ the most probable hypothesis H_{MAP}

~ the biggest a posteriori probability

$$H_{MAP} = H_J \ ; \ P(H_J|E) = \max_t \frac{P(E|H_t) P(H_t)}{P(E)}$$

→ neglect the denominator: $P(E) = \sum_t P(E|H_t) P(H_t)$

$$H_{MAP} = H_J \Leftrightarrow P(E|H_J) P(H_J) = \max_t (P(E|H_t) P(H_t))$$

Bayesian Classification (3)

Example:

Shall we provide credit to a client with a high income?

- $P(\text{BORROW}) = 0.667$
 - $P(\text{REJECT}) = 0.333$
 - $P(\text{HIGH_INCOME} | \text{BORROW}) = 0.91$
 - $P(\text{HIGH_INCOME} | \text{REJECT}) = 0.12$
 - $P(\text{LOW_INCOME} | \text{BORROW}) = 0.09$
 - $P(\text{LOW_INCOME} | \text{REJECT}) = 0.88$
-
- $P(\text{HIGH_INCOME} | \text{BORROW}) P(\text{BORROW}) = 0.607$
 - $P(\text{HIGH_INCOME} | \text{REJECT}) P(\text{REJECT}) = 0.040$
- $H_{\text{MAP}} = \text{BORROW}$

Naïve Bayesian Classifier

- Assumption:

- the respective events $E_1, \dots, E_K$ are **conditionally independent** subject to the validity of the hypothesis H
- in real-world tasks, this assumption is rarely fulfilled
→ therefore called „naïve“

→ evaluation of the a posteriori probability of the hypothesis H subject to the validity of all $E_1, \dots, E_K$

$$P(H \mid E_1, \dots, E_K) = \frac{P(E_1, \dots, E_K \mid H) P(H)}{P(E_1, \dots, E_K)}$$

Naïve Bayesian Classifier (2)

→ evaluation of the a posteriori probability of the hypothesis subject to the validity of all $E_1, \dots, E_K$ as:

$$P(H \mid E_1, \dots, E_K) = \frac{P(H)}{P(E_1, \dots, E_K)} \prod_{k=1}^K P(E_k \mid H)$$

→ for classification by means of the naive Bayesian classifier, we will look for the hypothesis with the biggest a posteriori probability H_{MAP}

$$H_{MAP} = H_J \Leftrightarrow P(H_J) \prod_{k=1}^K P(E_k \mid H_J) = \max_t \left(P(H_t) \prod_{k=1}^K P(E_k \mid H_t) \right)$$

Naïve Bayesian Classifier (3)

- $P(H_t) = P(\text{class_}t) =$
 $= \text{Nr_of_patterns_from_}t / \text{Nr_of_all_patterns}$
- $P(E_k | H_t) = P(\text{patterns_from_}t_matching_E_k) =$
 $= \frac{\text{Nr_of_patterns_from_}t_matching_E_k}{\text{Nr_of_patterns_from_}t}$

Naïve Bayesian Classifier (4)

+ possibility to classify also incompletely described patterns

- zero a posteriori probability of $P(E_k | H_t)$ in the case of no patterns in the training set (for E_k)

or underestimated terms $P(E_k | H_t)$ in the case of a low mutual incidence of E_k and H_t

Example:

training data for the Bayesian classifier

Client	Income	Account	Gender	Unemployed	Credit
1	HIGH	HIGH	FEMALE	NO	YES
2	HIGH	HIGH	MALE	NO	YES
3	LOW	LOW	MALE	NO	NO
4	LOW	HIGH	FEMALE	YES	YES
5	LOW	HIGH	MALE	YES	YES
6	LOW	LOW	FEMALE	YES	NO
7	HIGH	LOW	MALE	NO	YES
8	HIGH	LOW	FEMALE	YES	YES
9	LOW	MIDDLE	MALE	YES	NO
10	HIGH	MIDDLE	FEMALE	NO	YES
11	LOW	MIDDLE	FEMALE	YES	NO
12	LOW	MIDDLE	MALE	NO	YES

Bayesian classifier – an example

- Apriori probability of various values of the target attribute **CREDIT**:
 - $P(\text{Credit}(\text{YES})) = 8/12 = 0.667$
 - $P(\text{Credit}(\text{NO})) = 4/12 = 0.333$
 - $P(\text{Account}(\text{Middle}) | \text{Credit}(\text{YES})) = 2/8 = 0.25$
 - $P(\text{Account}(\text{Middle}) | \text{Credit}(\text{NO})) = 2/4 = 0.5$
 - $P(\text{Unemployed}(\text{NO}) | \text{Credit}(\text{YES})) = 5/8 = 0.625$
 - $P(\text{Unemployed}(\text{NO}) | \text{Credit}(\text{NO})) = 1/4 = 0.25$
 - ...

Bayesian classifier – an example

Example (continued):

→ For a client who applies for credit, has a middle balance on his account and is not unemployed, we will compute:

$$P(\text{Credit}(\text{YES})) \cdot P(\text{Account}(\text{Middle}) | \text{Credit}(\text{YES})) \cdot \\ \cdot P(\text{Unemployed}(\text{NO}) | \text{Credit}(\text{YES})) = 0.1042$$

$$P(\text{Credit}(\text{NO})) \cdot P(\text{Account}(\text{Middle}) | \text{Credit}(\text{NO})) \cdot \\ \cdot P(\text{Unemployed}(\text{NO}) | \text{Credit}(\text{NO})) = 0.0416$$

→ The client will be assigned to class **Credit (YES)**

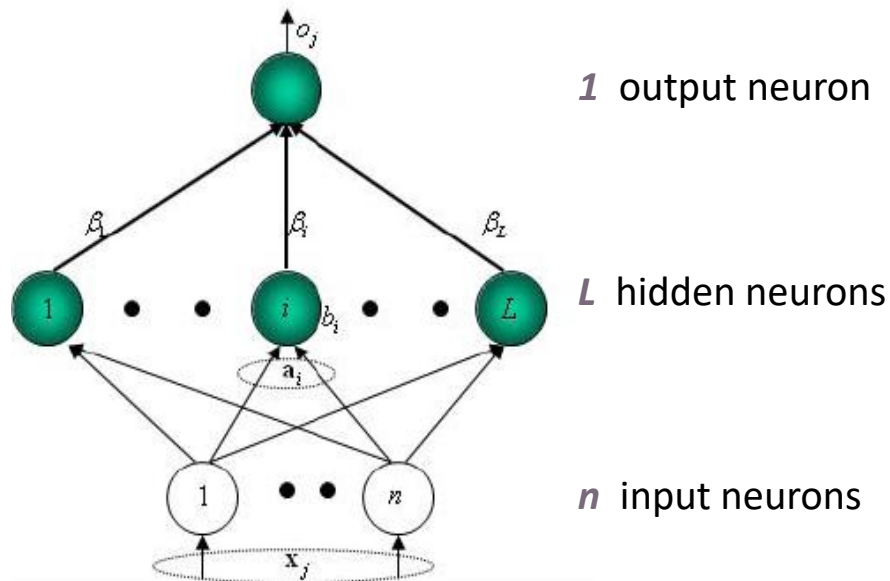
Data Mining

EXTREME LEARNING MACHINES

BY: IVETA MRÁZOVÁ

ELM-networks:

Extreme Learning Machines (G.-B. Huang)



A feedforward network with one hidden layer

- **Output of hidden neurons:**

$$h(x) = [G(a_1, b_1, x), \dots, G(a_L, b_L, x)]$$

- a_i : the weight vector of the i -th neuron
- b_i : bias of the i -th neuron

- **Sigmoid:**

$$G(a_i, b_i, x) = g(a_i x + b_i)$$

- **RBF:**

$$G(a_i, b_i, x) = g(b_i \|x - a_i\|)$$

- **Output of the network:**

$$f_L(x) = \sum_{i=1}^L \beta_i G(a_i, b_i, x)$$

- β_i : the weight between the i -th hidden and the output neuron

ELM-networks: a new approach to learning

Training without iterative adjustments of the model:

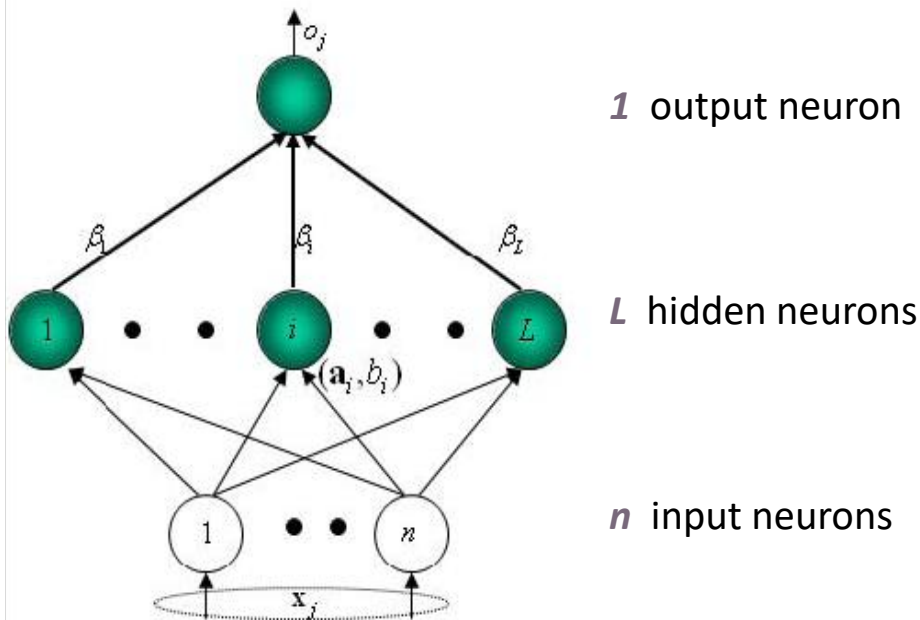
- Let us have a piecewise continuous function g . If it would be possible to approximate a continuous function $f(x)$ by means of a feedforward neural network with a hidden layer of adjustable neurons that would compute the function g , it would not be necessary to adapt the parameters of the hidden neurons in such a network.
- The parameters of all its hidden neurons can be randomly generated, even without the knowledge of the training data.

For each continuous function f and a randomly generated sequence $\{(a_i, b_i)_{i=1}^L\}$ it holds with probability one:

$$\lim_{L \rightarrow \infty} \|f(x) - f_L(x)\| = \lim_{L \rightarrow \infty} \|f(x) - \sum_{i=1}^L \beta_i G(a_i, b_i, x)\| = 0$$

subject to setting the values of β_i in such a way, that they minimize $\|f(x) - f_L(x)\|$, $i = 1, \dots, L$.

ELM-networks: a general training problem



A feedforward network with one hidden layer of neurons with a piecewise continuous transfer function $G(a_i, b_i, x)$

The mathematical model:

□ for N different randomly chosen training patterns $(x_i, t_i) \in R^n \times R^n$, it is possible to model the feedforward network with L hidden neurons that have the transfer function $G(a_i, b_i, x)$, and are organized within a single hidden layer, as:

$$\sum_{i=1}^L \beta_i G(a_i, b_i, x_j) = t_j; \quad j = 1, \dots, N$$

- (a_i, b_i) : parameters of the i -th hidden neuron
- β_i : the weight between the i -th hidden and the output neuron

ELM-networks: a general training problem

The mathematical model:

- $\sum_{i=1}^L \beta_i G(a_i, b_i, x_j) = t_j ; j = 1, \dots, N$ is equivalent to $\mathbf{H} \boldsymbol{\beta} = \mathbf{T}$, where

$$\mathbf{H} = \begin{bmatrix} h(x_1) \\ \vdots \\ h(x_N) \end{bmatrix} = \begin{bmatrix} G(a_1, b_1, x_1) & \dots & G(a_L, b_L, x_1) \\ \vdots & \ddots & \vdots \\ G(a_1, b_1, x_N) & \dots & G(a_L, b_L, x_N) \end{bmatrix}_{N \times L}$$

$$\boldsymbol{\beta} = \begin{bmatrix} \beta_1^T \\ \vdots \\ \beta_L^T \end{bmatrix}_{L \times m} \quad \text{and} \quad \mathbf{T} = \begin{bmatrix} t_1^T \\ \vdots \\ t_N^T \end{bmatrix}_{N \times m}$$

- $\mathbf{H}$ denotes the output matrix of the hidden layer of the neural network; the i -th column of $\mathbf{H}$ is the output of the i -th hidden neuron for the inputs $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N$.

ELM-networks: the training algorithm

- **Input:**

Training set $\mathfrak{X} = \{(x_j, t_j) \mid x_j \in R^n, t_j \in R^m, j = 1, \dots, N\}$,
transfer function of hidden neurons $G(a, b, x)$ and the
number of hidden neurons L

- **Algorithm:**

1. Choose randomly the parameters of hidden neurons $(a_i, b_i), 1 \leq i \leq L$
2. Compute the matrix of the outputs in the hidden layer H
3. Compute the weights for the output neurons $\beta: \beta = H^+ T$

- H^+ denotes the Moore-Penrose pseudoinverse matrix for the output matrix H from the hidden layer of the neural network

ELM-networks: the training algorithm

To get a better solution:

- more stable with improved generalization
- add a positive value $1/C$ to the elements on the diagonal of $H^T H$
- draw the solution from the relationship:

$$f(x) = h(x)\beta = h(x)(H^T H)^{-1} H^T T \rightarrow$$

$$\rightarrow h(x)(I/C + H^T H)^{-1} H^T T$$

ELM-networks: the training algorithm

Advantages:

- a simple non-iterative procedure consisting of three steps (many traditional training algorithms are considerably more complex)
- very fast and straight-forward training process (it is not necessary to solve the tasks concerning local minima, appropriate parameter initialization, overtraining, etc.)
- the parameters of the hidden neurons a_i and b_i are independent of the training data and themselves, too
- ELM-networks can generate the parameters of the hidden neurons already before presenting the training data (conventional training techniques must „see“ the training data before they set the parameters of the hidden neurons)
- ELM-networks can be used with any bounded piecewise continuous non-constant transfer function (gradient training algorithm work with smooth transfer functions only)